

HR AI AGENT BOOT CAMP · BUILD ALONG

DART API부터 배포까지 직접 만들어가기

OpenDART 키 발급 → .env → Claude Code 프롬프트 → 검증 → GitHub → Vercel

4시간 실습 · 완성 화면보다 만드는 과정을 따라가는 강의

오늘 수강생이 실제로 하는 일

빈 Starter를 열고, 프롬프트로 기능을 한 단계씩 붙인다



매 단계에서 “프롬프트 → 변경 파일 → 실행 결과”를 직접 확인합니다.

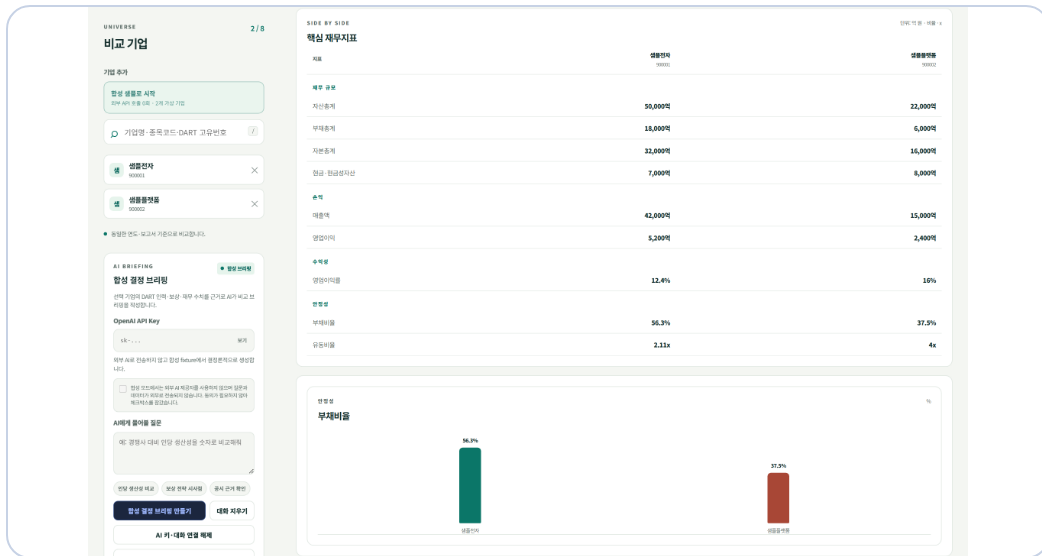
완성본은 목표 확인용 2분

완성 화면은 한 번만 보고 바로 닫는다

이 화면을 설명하는 것이 오늘의 수업이 아닙니다. 지금부터 이 결과가 생기는 순서를 재현합니다.

오늘의 핵심은 변화 과정

어떤 프롬프트를 넣었는가
어떤 파일이 바뀌었는가
무엇으로 통과를 확인했는가



최종 화면은 마지막 검증에서 다시 확인

4시간 실습 지도

설명보다 입력·수정·검증 시간이 더 길다

13:00-13:45

계정·API 키
.env·서버·Claude

14:00-14:45

기본 화면과
선택 동작 구현

15:00-15:45

DART·지표·AI
프롬프트 구현

16:00-16:50

GitHub·Vercel
배포와 재현

수강생 증거물

완성 화면보다 이 네 가지 기록을 제출한다

① 환경 설정 증거

키 값은 가리고 health의 **api_key_configured:**
true만 캡처

② 프롬프트와 변경 증거

Claude 요청문, 변경 파일 목록, diff 확인

③ 기능 검증 증거

기업 검색·실데이터 비교·결측 표시·AI 근거

④ 배포 증거

GitHub commit과 Vercel root·health URL

OpenDART 키와 개발 환경 준비

먼저 키를 발급받고 .env에 넣어 서버가 읽는 것까지 확인합니다.

수업 전 준비

네 계정과 편집·실행 도구를 먼저 확인한다

OpenDART

인증키 발급용 이메일과 비밀번호

Claude Code

로그인 가능한 Claude 계정

GitHub · Vercel

저장소 생성과 Import 권한

로컬 도구

VS Code · Python 3.11+ · Git · Chrome/Edge

STEP 01 · Starter 복사

Starter를 복사하고 기준 커밋을 만든다



```
PS C:\work> Copy-Item -Recurse
C:\class\workshop_starter C:\work\dart-
hr-agent
PS C:\work> cd C:\work\dart-hr-agent
PS ...Wdart-hr-agent> git init
PS ...Wdart-hr-agent> git add .
PS ...Wdart-hr-agent> git commit -m "Save
workshop starter"
[main (root-commit) ...] Save
workshop starter
```

1**Starter 복사**

원본은 복구용으로 그대로 보존

2**새 Git 시작**

개인 작업 폴더에서 초기화

3**기준 커밋**

이후 Claude 변경이 diff로 보임

STEP 02 · Claude Code 확인

Claude Code를 실행하고 프로젝트 폴더 안인지 확인한다

```
PS ...Wdart-hr-agent> claude --version  
2.x.x
```

```
PS ...Wdart-hr-agent> claude
```

```
┌ Claude Code
```

```
| C:\work\dart-hr-agent |
```

```
> █
```

처음 실행이라면

브라우저 로그인 안내를 따라 인증

문제가 있다면

claude doctor로 설치 상태 확인

통과

프로젝트 경로와 프롬프트 입력창이 보임

STEP 03 · OpenDART 접속

OpenDART에서 인증키 신청 메뉴를 연다

1. opendart.fss.or.kr 접속

상단 메뉴에서 인증키 신청/관리 →
인증키 신청

2. 개인 또는 기업 선택

강의용 개인 신청은 계정 신청 완료 후 즉시
발급 안내

 opendart.fss.or.kr

OPEN DART

오픈API 소개 인증키 신청/관리 개발가이드

신청 구분	☒ 개인 ☐ 기업
이메일	본인 이메일 입력
API 사용환경	☒ 웹 ☐ 앱 ☐ 모두
API 사용용도	교육용 HR Analytics 실습

등록

STEP 04 · 인증키 신청

약관 동의와 필수 항목을 입력하고 등록한다

① 이용약관

필수 약관과 개인정보 수집
항목 확인

② 공통정보

사용환경·사용용도·확인
URL 입력

③ 등록

메일과 인증키 관리에서
결과 확인

강의 전달까지 완료

개인 신청은 즉시 발급 안내, 기업 신청은 담당자 승인 1~2영업일 안내가 있습니다. 수업 현장에서 새 기업 계정을 만들지 않습니다.

STEP 05 · 키 확인

인증키 관리에서 키를 복사하되 화면에는 남기지 않는다

 opendart.fss.or.kr**OPEN
DART**오픈API 소개 인증키 신청/
관리 개발가이드

인증키 상태	사용 가능
API 인증키	
신청일	수업 전 발급 완료
이용현황	정상

복사 직후

.env에 한 번만 붙여넣기

화면 공유

키 입력 구간은 잠시 중지

캡처

키 값·요청 URL·헤더를 찍지 않음

금지

Claude 대화창이나 소스 코드에 키 붙여넣기

STEP 06 · .env 만들기

.env.example을 복사해 로컬 전용 .env를 만든다



```
PS ...Wdart-hr-agent> Copy-Item  
.env.example .env
```

```
PS ...Wdart-hr-agent> code .env
```

```
# 예시 파일을 복사해  
# 변수 이름을 그대로 유지
```

1

예시 파일 복사

변수 이름을 틀리지 않게 유지

2

.env 열기

프로젝트 루트 파일인지 확인

3

한 줄만 수정

OPENDART_API_KEY= 오른쪽

STEP 07 · 키 입력

인증키는 .env에만 넣고 저장한다

Visual Studio Code · .env

EXPLORER

▼ dart-hr-agent

▸ api

▸ static

● .env

.env.example

.gitignore

server.py

OpenDART API 인증키 (40자리)

OPENDART_API_KEY=[여기에_본인의_40자리_인증키]

서버 공용 AI 키는 선택 사항

OPENAI_API_KEY=

OPENAI_MODEL=gpt-5.6-luna

STEP 08 · Git 제외 확인

서버를 켜기 전에 .env가 Git에서 제외되는지 확인한다



```
PS ...> git check-ignore .env
.env
```

```
PS ...> git ls-files .env
# 아무것도 나오지 않아야 함
```

```
PS ...> git status --short
# .env가 목록에 없음
```

통과

check-ignore는 .env 출력, ls-files는 빈 결과

중단

git ls-files .env가 값을 반환하면 커밋 전에 추적 해제

캡처

명령 결과만 찍고 .env 편집기는 닫기

STEP 09 · 서버 실행

Python 서버를 실행하고 실제 로컬 주소를 연다



```
PS ...> python server.py
```

```
DART HR Workshop Starter:
```

```
http://127.0.0.1:8765
```

```
OpenDART key configured:
```

```
True
```

브라우저가 자동으로 열리지 않으면

127.0.0.1:8765

Starter는 기본 포트 8765를 사용합니다. 포트 충돌이
나면 사용 중인 프로세스를 종료하거나 PORT
환경변수로 바꿉니다.

STEP 10 · 키 연결 확인

키 값 대신 “설정됨” 상태만 확인한다



```
PS ...> $h = Invoke-RestMethod `
    http://127.0.0.1:8765/api/health
```

```
PS ...> $h.app.id
kr.opendart.dart-hr-workshop-starter
```

```
PS ...> $h.api_key_configured
True
```

True면 통과

서버가 .env에서 키를 읽었음

False면

파일명·위치·변수명·서버 재시작 확인

절대 확인하지 않을 것

키 문자열 자체를 출력하거나 공유

STEP 11 · 프롬프트 입력 방법

전체 프롬프트를 붙여넣고 Enter를 누른다



> 이 저장소를 먼저 분석해줘.
아직 코드는 수정하지 마.

1. 실행 진입점
2. 화면 파일
3. OpenDART 호출 위치
4. 지표 계산 위치
5. Vercel 배포 파일

각 파일의 역할과 다음에 수정할 순서를 설명해줘.

1**프롬프트 전체 복사**

코드 블록 안의 문장만 복사

2**Claude 입력창에 붙여넣기**

API 키는 붙여넣지 않음

3**Enter 한 번**

응답이 끝날 때까지 추가 입력 금지

STEP 12 · 첫 Claude 프롬프트

첫 요청은 코드를 만들라는 말이 아니라 “읽어달라”는 요청이다

PROMPT 01 · 저장소 분석

CLAUDE CODE

이 저장소를 먼저 분석해줘. 아직 코드는 수정하지 마.

1. 실행 진입점
2. 화면 파일
3. OpenDART 호출 위치
4. 지표 계산 위치
5. Vercel 배포 파일

각 파일의 역할과 다음에 수정할 순서를 설명해줘.

붙여넣기 → Enter · 수정 승인 금지

Claude가 작업하는 동안

읽는 파일과 실행하려는 명령을 화면에서 확인한다



Claude: 먼저 저장소 구조를 확인하겠습니다.

```
Read README.md
Read server.py
Read static/index.html
Read static/app.js
Read api/index.py
Read vercel.json
```

코드는 수정하지 않았습니다.

허용

파일 읽기 · 검색 · 실행 방법 확인

멈춤

첫 요청인데 Edit·Write·삭제를 시도함

확인

현재 프로젝트 경로가 맞는지 항상 보기

Claude 응답 확인

답변에서 다섯 파일의 역할을 찾아 표시한다

파일	수강생이 확인할 역할	다음 단계
static/index.html	화면 구조	UI 프롬프트
static/app.js	선택·요청·렌더링	DART 연결
server.py	.env와 API 경로	키·데이터
api/index.py	Vercel 진입점	배포
vercel.json	경로 라우팅	배포 검증

프롬프트 작성 규칙

목적·입력·규칙·완료 기준을 한 번에 준다

목적

무엇을 만들 것인가

입력

기업·연도·보고서

규칙

키·결측·기존 구조

완료 기준

화면·명령·테스트

“예쁘게 만들어줘”가 아니라 검증 가능한 상태를 요청합니다.

Claude Code로 기본 화면 만들기

화면 뼈대 → 기업 선택 → 기간 선택 순서로 후속 기능을 붙입니다.

STEP 13 · UI 계획 요청

구현 전에 변경 파일과 완료 기준부터 받는다

PROMPT 02 · 계획만 작성

CLAUDE CODE

HR Analytics 기본 화면을 만들려고 해.
아직 수정하지 말고 계획만 작성해줘.

필수 요소: 기업 검색, 선택 기업 2개, 기준연도,
보고서 종류, 비교 버튼, 결과 영역.

변경 파일·구현 순서·완료 기준을 표로 보여줘.

아직 코드를 수정하지 않도록 명시

Claude 계획 검토

계획이 이번 범위를 넘으면 구현 전에 줄인다

이번 단계에 포함

화면 뼈대와 빈 상태

다음 프롬프트로 미룸

선택 동작 · 기간 · 실제 API · AI

변경 파일

`static/index.html` · `static/styles.css`

계획이 맞으면 입력

“첫 단계만 진행해줘.”

한 번에 전체 앱을 만들지 않습니다.

STEP 14 · 첫 화면 구현

서비스 제목과 입력 영역만 만드는 프롬프트를 넣는다

PROMPT 03 · 화면 뼈대

CLAUDE CODE

계획의 첫 단계만 구현해줘.

static/index.html과 static/styles.css에

- 서비스 제목과 한 줄 소개
- 기업 검색 입력창
- 선택 기업 영역
- 결과가 아직 없다는 안내

를 만들어줘. 실제 API와 선택 동작은 아직 넣지 마.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

수정 승인 화면

파일 경로와 수정 내용을 보고 승인한다



Claude wants to edit:

`static/index.html`

+ service header

+ company search input

+ empty result area

`static/styles.css`

+ layout and input styles

Make this edit?

> 1. Yes 2. No

Yes 선택

계획한 파일과 내용이 맞을 때

No 선택

API·AI·새 라이브러리가 섞였을 때

승인 후

Claude의 완료 문구가 아니라 diff 확인

STEP 15 · 첫 변경 확인

파일이 만들어진 뒤 terminal에서 diff를 본다



```
PS ...> git status --short
```

```
M static/index.html
```

```
M static/styles.css
```

```
PS ...> git diff --stat
```

```
static/index.html | 28 +++++
```

```
static/styles.css | 54 ++++++++
```

```
PS ...> git diff
```

요청한 요소

제목 · 소개 · 검색창 · 선택 영역 · 빈 상태

아직 없어야 할 것

실제 API 호출 · AI 호출 · API 키

STEP 16 · 기능 추가 프롬프트

같은 대화에서 기업 선택 동작만 추가한다

PROMPT 04 · 기업 선택 추가

CLAUDE CODE

방금 만든 화면에 기업 선택 동작을 추가해줘.

- 검색 결과를 임시 샘플 3개로 보여준다.
- 최대 2개를 추가·삭제할 수 있다.
- 중복 선택을 막는다.
- 선택 수를 0/2처럼 표시한다.

기존 화면 스타일과 파일 구조를 유지해줘.

기존 결과를 유지한 채 한 기능만 추가

기업 선택 결과 확인

프롬프트 한 번으로 화면 상태가 어떻게 바뀌었는지 비교한다

BEFORE

검색창만 있음

입력해도 결과·선택 상태 없음

AFTER PROMPT 04

검색 결과와 0/2 선택

추가·삭제·중복 방지 동작 확인

STEP 17 · 다음 기능 추가

기업 선택은 유지하고 기간과 비교 버튼만 붙인다

PROMPT 05 · 기간과 비교 버튼

CLAUDE CODE

기업 선택 기능은 그대로 두고 다음 기능만 추가해줘.

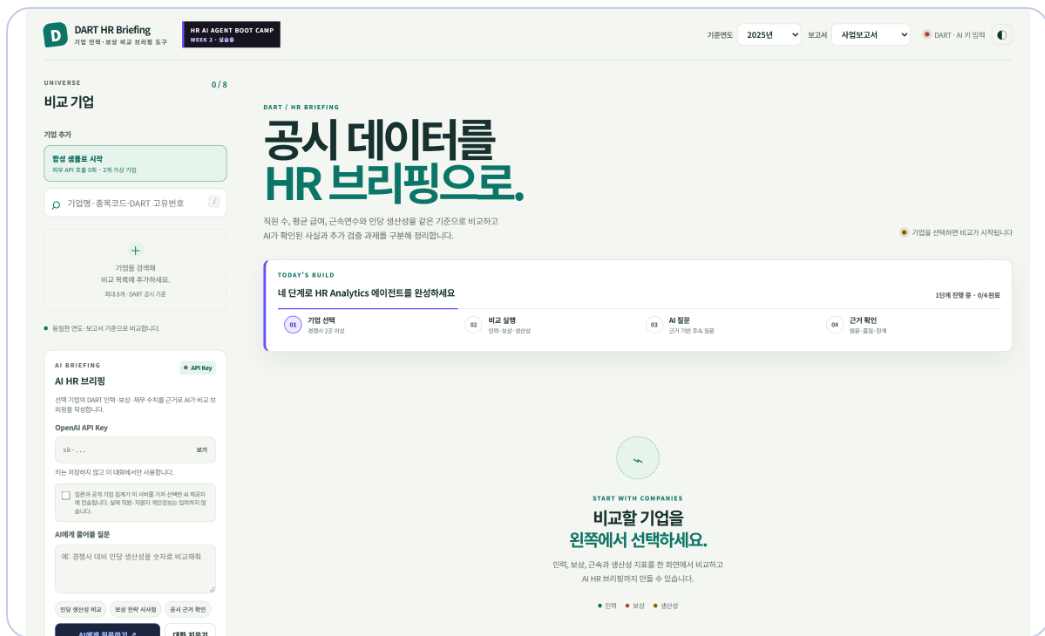
- 기준연도 선택
- 사업·반기·분기보고서 선택
- 인력·보상 비교 버튼
- 조건이 바뀌면 이전 결과 비우기

아직 OpenDART와 AI는 호출하지 마.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

STEP 18 · 브라우저 검증

서버를 다시 열고 추가된 기능을 손으로 조작한다



기업 검색 입력

검색 결과와 0/2 상태

선택 추가·삭제

중복 없이 두 기업 표시

조건 변경

이전 결과가 남지 않음

직접 확인: 입력창 · 선택 목록 · 연도 · 보고서 · 비교 버튼

문제가 생겼을 때

오류 화면을 그대로 보여주고 작은 수정만 요청한다

PROMPT 06 · 오류 수정

CLAUDE CODE

현재 화면과 터미널 오류를 읽어줘.
원인 후보를 3개 이내로 좁힌 뒤 가장 작은 수정부터 적용해줘.
수정 전 재현 조건, 수정 파일, 수정 후 확인 결과를 정리해줘.
관련 없는 파일은 바꾸지 마.

오류 복사 → 원인 축소 → 작은 수정 → 같은 동작 재검증

MODULE 02 체크포인트 · 14:45

프롬프트 세 번으로 화면이 자라는 과정을 설명해야 통과한다

PROMPT 03

화면 뼈대 생성

PROMPT 04

기업 선택 동작 추가

PROMPT 05

기간·보고서·비교 버튼 추가

남겨야 할 증거

전체 프롬프트

Claude 수정 승인 화면

```
git diff --stat
```

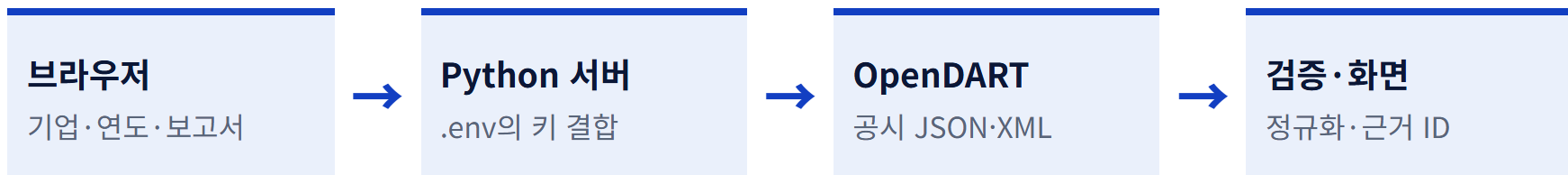
브라우저 Before/After

OpenDART 연결과 HR 지표·AI 구현

API 호출도 같은 순서입니다. 먼저 설계하고, 한 경로씩 붙이고, 실제
응답으로 검증합니다.

STEP 19 · 데이터 흐름

브라우저에 키를 보내지 않고 서버가 OpenDART를 호출한다



키는 OpenDART HTTPS 요청에만 포함되고 브라우저로 돌아오지 않습니다.

사용할 OpenDART 데이터

세 API와 요청값의 차이를 먼저 고정한다

목적	API·파일	핵심 결과
기업 검색	corpCode.xml	corp_code 8자리
재무정보	fnltdSinglAcnt.json	매출액·영업이익
직원현황	empSttus.json	직원수·평균급여

공통: crtfc_key · 기업목록 ZIP: 키만 · 재무/직원: corp_code + bsns_year + reprt_code

STEP 20 · API 설계 요청

실제 코드를 쓰기 전에 호출·결측·검증 계획을 받는다

PROMPT 07 · OpenDART 연결 계획

CLAUDE CODE

선택 기업·연도·보고서로 직원수, 평균급여,
매출액, 영업이익을 조회하려고 해.

아직 수정하지 말고 사용할 OpenDART API, 요청값,
서버 경로, 변경 파일, 결측 처리, 검증 방법을 설계해줘.

계획만 작성 · 엔드포인트와 요청값을 사람이 대조

STEP 21 · 서버 키 경계 검증

.env를 읽는 위치와 health 응답이 안전한지 Claude로 검사한다

PROMPT 08 · 환경변수 경계

CLAUDE CODE

현재 Starter의 OPENDART_API_KEY 처리 경계를 점검해줘.
계약을 만족하지 않는 부분만 수정해줘.

- 실제 키를 코드·로그·응답에 넣지 않는다.
- .env.example에는 변수 이름만 둔다.
- .env가 Git에서 제외되는지 확인한다.
- /api/health에는 설정 여부만 반환한다.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

STEP 22 · 기업 검색 구현

화면 검색어를 DART corp_code로 바꾸는 경로를 붙인다

PROMPT 09 · 기업 검색

CLAUDE CODE

기업 검색창을 /api/companies에 연결해줘.
두 글자 이상 입력하면 검색하고 기업명·종목코드를 보여줘.
선택하면 corp_code를 상태에 저장해줘.
검색 중·결과 없음·오류를 구분하고 중복 선택을 막아줘.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

API 연결 1 · 브라우저 코드

검색어를 우리 서버의 /api/companies로 보낸다



```
// static/app.js
const query = companySearch.value.trim();

const response = await fetch(
  `/api/companies?q=${encodeURIComponent(query)}`
);

const payload = await response.json();
renderSearchResults(payload.companies || []);
```

브라우저가 보내는 것

검색어 q=삼성 전자

브라우저가 보내지 않는 것

OpenDART 인증키

확인할 파일

static/app.js

API 연결 2 · 서버 경로

Python 서버가 검색어를 받고 기업 목록을 반환한다



```
# server.py
if parsed.path == "/api/companies":
    query = parse_qs(parsed.query).get("q", [""])[0]
    companies = search_companies(query)
    self.send_json({"companies": companies})
    return
```

입력

/api/companies?q=삼성전자

처리

search_companies(query)

출력

기업명·종목코드·corp_code JSON

API 연결 3 · OpenDART 요청

키를 붙여 받은 ZIP을 서버에서 XML로 푼다



```
# server.py
import json
from io import BytesIO
from zipfile import ZipFile

def dart_request(endpoint, params, *, binary=False):
    if not API_KEY:
        raise DARTError("키가 설정되지 않았습니다")
    query = {**params, "crtfc_key": API_KEY}
    url = f"{DART_BASE}/{endpoint}?{urlencode(query)}"
    with urlopen(url, timeout=10) as response:
        return response.read() if binary else json.load(response)

zip_bytes = dart_request("corpCode.xml", {}, binary=True)
with ZipFile(BytesIO(zip_bytes)) as archive:
    corp_xml = archive.read("CORPCODE.xml")
```

연결 경계

브라우저 → 우리 서버: 검색어만

우리 서버 → OpenDART: 인증키를 HTTPS
요청에 결합

우리 서버: ZIP 해제·XML 파싱

우리 서버 → 브라우저: 정리된 JSON만

키가 보이면 실패

브라우저 Network·응답 JSON·로그에 키가
없어야 함

API 연결 4 · 응답 확인

브라우저 주소창에서 우리 API의 JSON을 먼저 확인한다

```
GET http://127.0.0.1:8765/  
api/companies?q=삼성전자  
  
{  
  "companies": [  
    {  
      "corp_name": "삼성전자",  
      "stock_code": "005930",  
      "corp_code": "00126380"  
    }  
  ]  
}
```

1**HTTP 200**

우리 서버 경로가 열림

2**companies 배열**

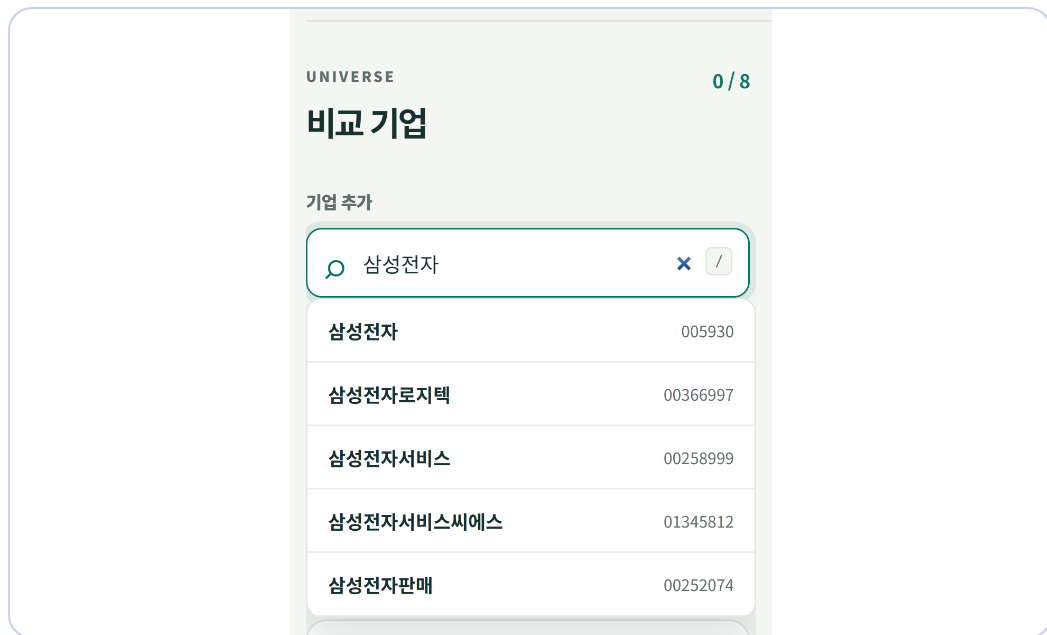
화면이 사용할 JSON 계약

3**키 문자열 없음**

비밀이 서버 밖으로 나오지 않음

STEP 23 · 기업 검색 검증

검색 결과에서 회사명·종목코드·corp_code를 확인한다



입력

삼성전자 또는 종목코드

응답

법인명 · 종목코드 · 8자리 식별자

예외

검색 중 · 결과 없음 · API 오류

실제 앱 조작 캡처 · 키 문자열은 화면에 없음

STEP 24 · 재무·직원 데이터 구현

두 API 결과를 같은 corp_code와 기간으로 결합한다

PROMPT 10 · 핵심 데이터 수집

CLAUDE CODE

선택 기업의 corp_code, 연도, 보고서 코드로
재무정보와 직원현황을 함께 조회해줘.

매출액·영업이익·직원수·평균급여를 corp_code로 결합하고,
키는 브라우저로 보내지 마. 로딩·결측·오류를 구분해줘.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

재무·직원 연결 1 · 브라우저 요청

선택한 corp_code와 기간을 우리 서버 경로로 보낸다

```
// static/app.js
const query = new URLSearchParams({
  corp_codes: selected.map(c =>
c.corp_code).join(","),
  year: "2024",
  report_code: "11011"
});

await Promise.all([
  fetch(`/api/financials?${query}`),
  fetch(`/api/people?${query}`)
]);
```

corp_codes

선택 기업의 DART 식별자

year

사업연도

report_code

11011 = 사업보고서

재무·직원 연결 2 · 서버가 OpenDART 호출

같은 기간 파라미터를 두 OpenDART API에 전달한다



재무정보

```
dart_request(  
    "fnlIttSinglAcnt.json",  
    {"corp_code": code,  
     "bsns_year": year,  
     "reprt_code": report_code}  
)
```



직원현황

```
dart_request(  
    "empSttus.json",  
    {"corp_code": code,  
     "bsns_year": year,  
     "reprt_code": report_code}  
)
```

재무·직원 연결 3 · 응답 결합

corp_code를 열쇠로 두 응답을 한 기업 행으로 합친다



```
{
  "example": "교육용 예시",
  "corp_code": "00126380",
  "year": "2024",
  "report_code": "11011",
  "financials": {
    "revenue": 3000000000000000,
    "operating_profit": 260000000000000
  },
  "people": {
    "employees_total": 120000,
    "average_salary": 130000000
  }
}
```

결합 기준

corp_code + year + report_code

숫자 주의

화면의 숫자는 구조 설명용 예시

다음 단계

실제 응답에 결측·단위·근거 규칙 적용

STEP 25 · 데이터 품질 규칙

값이 없을 때 0으로 보이면 분석이 틀어진다

PROMPT 11 · 결측과 근거

CLAUDE CODE

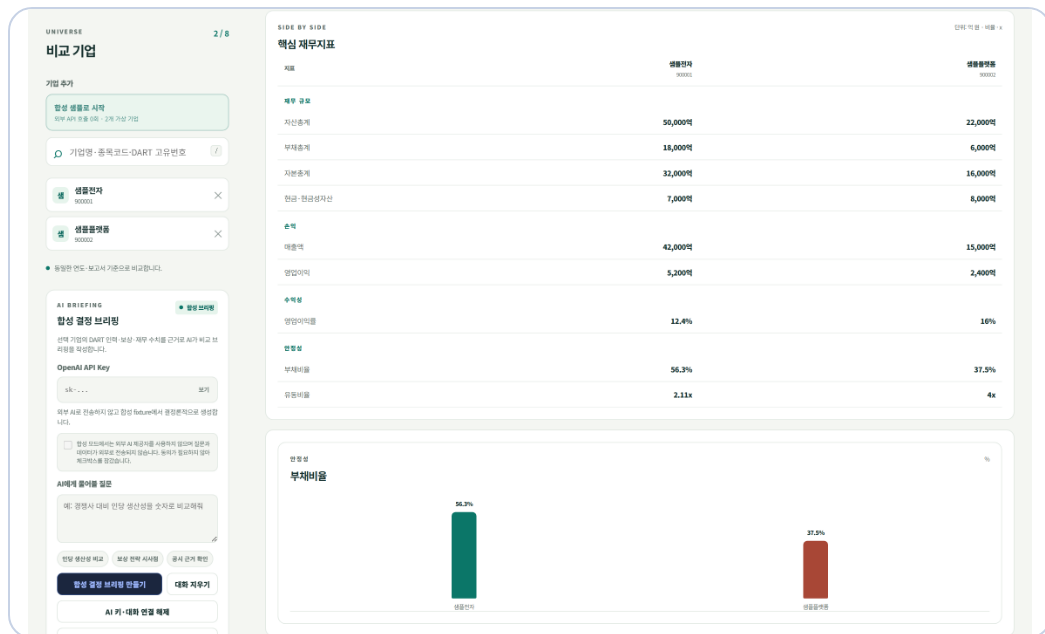
공시 데이터 품질 규칙을 적용해줘.

- 데이터 없음은 0으로 바꾸지 않는다.
- 실제 0과 결측을 화면에서 구분한다.
- 단위·기준연도·보고서·연결/별도 범위를 표시한다.
- 각 값에 evidence_id를 연결한다.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

STEP 26 · 실제 데이터 검증

비교 결과에서 값·단위·근거를 직접 확인한다



확인 1

두 기업 모두 같은 기간인가

확인 2

원자료 단위와 범위가 보이는가

확인 3

결과와 실제 0이 구분되는가

확인 4

DART 근거 링크가 기업에 맞는가

기업·연도·보고서 선택 → 비교 실행 → 결과 카드 확인

STEP 27 · HR 지표 계산

공시 원자료를 계산 함수로 바꾸는 프롬프트를 넣는다

PROMPT 12 · 파생지표

CLAUDE CODE

별도 함수로 다음 지표를 계산해줘.

1. 인당 매출 = 매출액 / 직원수
2. 인당 영업이익 = 영업이익 / 직원수
3. 직원수 증가율
4. 평균급여 증가율

결측·분모 0이면 계산하지 마.

tests/test_workforce_analytics.py에 정상값, 직원수 결측, 분모 0, 영업이익 음수 테스트를 만들고 실행 결과를 보여줘.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

STEP 28 · 계산 검증

정상값 하나보다 결측·0·음수 사례를 함께 본다



```
PS ...> python -m unittest `
    tests/test_workforce_analytics.py -v
```

```
test_revenue_per_employee ... ok
test_missing_employees ... ok
test_zero_denominator ... ok
test_negative_profit ... ok
```

```
Ran 4 tests · OK
```

사람이 계산기로 한 번 대조

매출액 ÷ 직원수

영업이익 ÷ 직원수

전년값이 있을 때만 증가율

단위가 원·천원·백만원 중 무엇인지 먼저 확인합니다.

STEP 29 · AI 입력 계약

AI에게 문서 전체가 아니라 검증된 숫자만 보낸다

PROMPT 13 · AI payload

CLAUDE CODE

AI에게 원문 전체를 보내지 말고 다음만 담은 payload를 만들어줘.

- 선택 기업과 기준연도
- 검증된 원자료
- 계산된 HR 지표
- evidence_ids
- 사용자 질문

전송 전에 스키마를 검증하고 키·개인정보는 제외해줘.

붙여넣기 → 제안 파일 확인 → 승인 → 검증

AI에게 전달되는 모양

질문과 숫자, 계산, 근거 ID가 한 묶음이 된다



```
{  
  "company": "A사",  
  "year": "2024",  
  "employees": 10000,  
  "revenue": 2000000000000000,  
  "revenue_per_employee": 20000000000,  
  "evidence_ids": ["FIN-A-2024", "EMP-A-2024"],  
  "question": "생산성 차이를 설명해줘"  
}
```

포함

공개 기업 집계·계산값·근거 ID

제외

API 키·개인정보·원문 전체

검증

전송 전 스키마와 질문 범위 확인

STEP 30 · AI 브리핑 구현

Provider·경로·키 수명까지 프롬프트에 적는다

PROMPT 14 · HR Analytics Assistant

CLAUDE CODE

HR Analytics Assistant를 구현해줘.

연결 계약:

- 브라우저는 POST /api/ai/briefing으로 검증된 payload만 보낸다.
- Provider는 OpenAI Responses API이며 모델은 OPENAI_MODEL을 사용한다.
- 개인 키는 password 입력 → X-OpenAI-API-Key 헤더 → 서버 요청 메모리에서만 사용한다.
- 개인 키가 없으면 서버의 OPENAI_API_KEY를 사용한다.
- 키를 localStorage·로그·응답·오류문에 저장하거나 출력하지 않는다.

답변을 확인된 사실, 계산 결과, 해석, 추가 확인 정보로 구분해줘.

공시만으로 인과관계를 단정하거나 개인 평가·채용·보상·감축을 권고하지 못하게 하고,
근거 ID와 한계를 표시해줘. 연결 실패·인증 실패·모델 실패도 구분해줘.

연결 계약 확인 → 구현 승인 → 네트워크·로그 검증

AI 연결 1 · 브라우저 요청

검증된 payload와 선택적 개인 키를 우리 서버로 보낸다



```
// static/app.js
const headers = {"Content-Type": "application/json"};
const personalKey = aiKey.value.trim();
if (personalKey) {
  headers["X-OpenAI-API-Key"] = personalKey;
}

const response = await fetch("/api/ai/briefing", {
  method: "POST",
  headers,
  body: JSON.stringify(validatedPayload)
});
```

개인 키가 있으면

password 입력값을 이 요청에만 사용

개인 키가 없으면

서버의 OPENAI_API_KEY 사용

저장 금지

localStorage·소스·응답·로그

AI 연결 2 · 서버 Provider 호출

서버가 Responses API를 호출해 답변만 반환한다

```
# server.py · POST /api/ai/briefing
header_key = self.headers.get("X-OpenAI-API-Key")
key = header_key or OPENAI_API_KEY
body = json.dumps({
    "model": OPENAI_MODEL,
    "input": briefing_prompt(payload)
}).encode("utf-8")
request = Request(
    "https://api.openai.com/v1/responses",
    data=body, method="POST",
    headers={"Authorization": f"Bearer {key}",
             "Content-Type": "application/json"}
)
result = json.load(urlopen(request, timeout=30))
answer = extract_output_text(result)
self.send_json({"answer": answer})
```

모델

OPENAI_MODEL=gpt-5.6-luna

키 수명

현재 HTTPS 요청 처리 동안만 메모리에 존재

반환

검증된 답변 텍스트와 근거 ID만

STEP 31 · 질문 실행

질문을 넣고 답변보다 근거 구조를 검사한다

[illegible]

직접 실행 캡처 · 공개 기업 집계 또는 합성 데이터만 사용

MODULE 03 체크포인트 · 15:45

프롬프트에서 실제 공시와 근거 있는 답변까지 이어져야 통과한다

데이터 통과

검색 → corp_code → 재무·직원 API → 결측·근거 표시

계산 통과

인당 지표와 결측·분모 0 테스트

AI 통과

payload → 서버 → Provider → 근거 있는 답변

증거 통과

PROMPT 07~14 · diff · 테스트 · Network

GitHub에 저장하고 Vercel에 배포하기

이제 만들어진 코드를 비밀값 없이 저장하고, 실제 URL에서 같은 동작을 재현합니다.

STEP 32 · 업로드 전 검사

git add 전에 .env와 키가 빠졌는지 다시 확인한다



```
PS ...> git status --short
PS ...> git check-ignore .env
.env
PS ...> git ls-files .env
# 빈 결과

PS ...> git grep -n OPENDART_API_KEY --
. "!.env"
.env.example:OPENDART_API_KEY
```

계속

예시 파일에는 변수 이름만 존재

즉시 중단

실제 키·개인정보·.env가 추적됨

수정 후

키를 폐기·재발급하고 검사부터 다시 실행

STEP 33 · GitHub 저장소 생성

github.com/new에서 빈 저장소를 만든다

1

Repository name

dart-hr-agent

2

Public 또는 Private

강의 제출 기준에 맞게 선택

3

초기화 옵션 끄기

README·gitignore·License 추가
안 함 github.com/new

Create a new repository

Owner	본인 계정
Repository	dart-hr-agent
Visibility	☒ Public ☐ Private
Initialize	☐ README ☐ .gitignore

[Create repository](#)

STEP 34 · 기능 커밋과 Push

기준 커밋 뒤의 변경만 검토해 원격 저장소로 보낸다

```
PS ...> git status --short
PS ...> git diff
PS ...> git add .
PS ...> git diff --cached --name-only
PS ...> git diff --cached

PS ...> git commit -m "Build DART HR agent"
PS ...> git branch -M main
PS ...> git remote add origin
https://github.com/ID/dart-hr-agent.git
PS ...> git push -u origin main
```

멈춰서 볼 지점

- ① 기준 커밋 이후의 실제 diff
- ② add 뒤 staged 파일
- ③ remote가 본인 저장소인지
- ④ push가 main → main인지

STEP 35 · GitHub 웹 검증

저장소 웹 화면에서 코드와 비밀 분리를 확인한다

README

실행 명령과 .env 생성 방법

배포 파일

api/index.py · vercel.json

최신 커밋

로컬 `git log -1`과 같은 해시

.env 없음

검색해도 실제 키 파일이 나오지 않음

STEP 36 · Vercel Import

vercel.com/new에서 방금 Push한 저장소를 가져온다

1

Add New Project

GitHub 연결 권한 확인

2

Repository 선택

dart-hr-agent Import

3

Project 설정

Framework: Other · Root: ./

 vercel.com/new

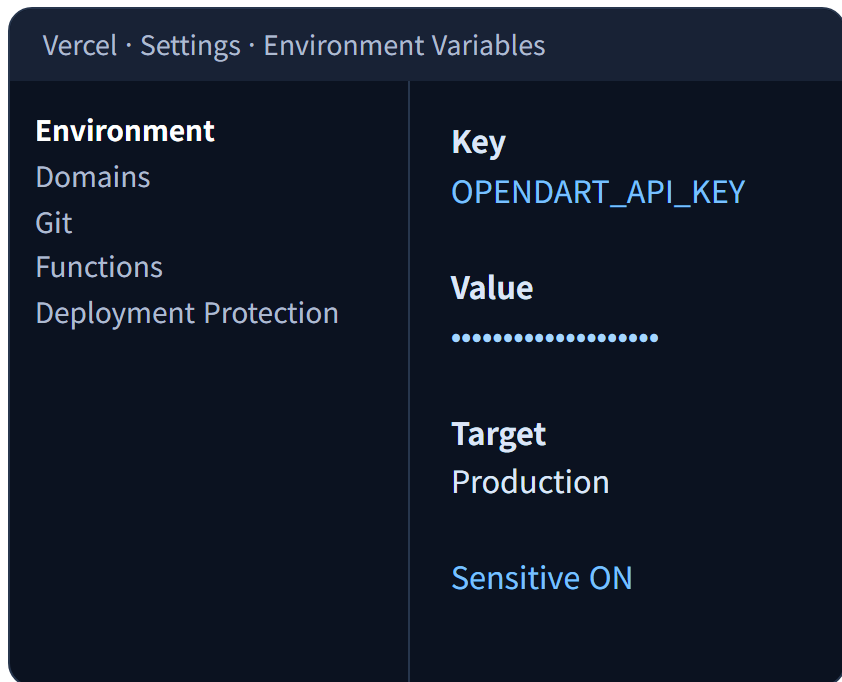
Import Git Repository

Repository	본인ID / dart-hr-agent
Framework	Other
Root Directory	./
Build / Output	기본값 유지

Import

STEP 37 · Vercel 환경변수

Production 환경에 OpenDART 키를 다시 등록한다



이름

OPENDART_API_KEY 정확히 입력

대상

Production 선택

보호

Sensitive 활성화

AI 키

개인 키 입력 방식이면 서버 등록 생략

STEP 38 · Deploy

Deploy를 누르고 Ready보다 Build Logs를 먼저 읽는다



Building에서 멈추면

Build Logs의 첫 번째 오류부터 읽고, 그 문장을 Claude Code에 붙여넣어 원인·수정 파일·재배포 방법을 요청합니다.

STEP 39 · 운영 URL 검증

배포 URL에서 로컬과 같은 순서로 다시 조작한다

1 루트 화면

`https://프로젝트.vercel.app/`

2 health

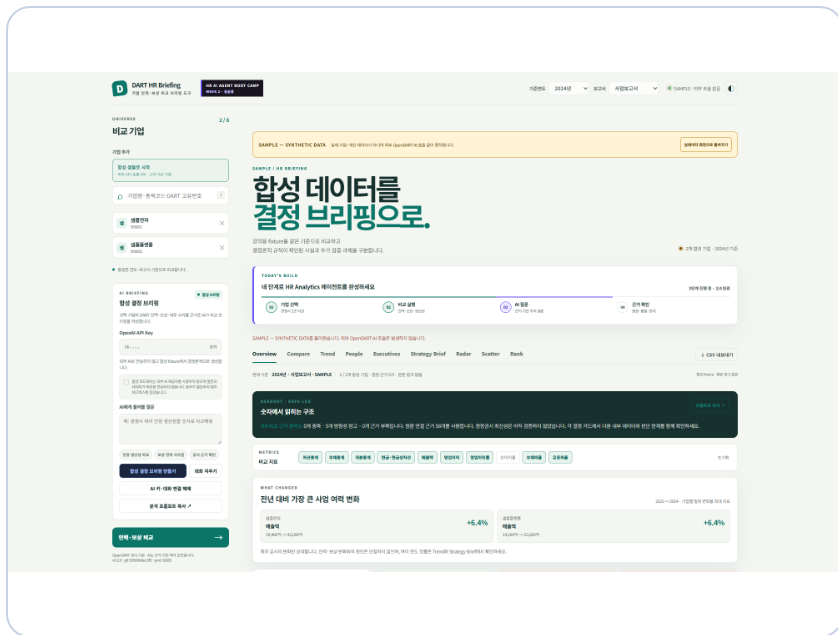
`/api/health · api_key_configured: true`

3 기업 검색

실제 OpenDART 검색 1회

4 비밀 확인

소스·응답·URL에 키가 없음



운영 화면과 health가 같은 배포 commit을 가리키는지 확인

STEP 40 · 수정 후 자동 재배포

코드를 고치면 같은 검증 루프를 한 번 더 돈다



환경변수만 바뀌어도 새 배포 또는 Redeploy가 필요합니다.

MODULE 04 체크포인트 · 16:40

저장소와 URL이 같은 코드를 보여줘야 통과한다

GitHub 증거

저장소 URL · 최신 commit · .env 없음

Vercel 증거

Production URL · Ready · 연결 commit

실행 증거

root · health · 기업 검색 · 비교

과정 증거

프롬프트 01~14 · diff · 테스트 결과

FINAL REPLAY

키를 넣고 프롬프트로 만들고 검증해서 배포했다

OpenDART 인증키 → .env → Claude Code → DART·HR·AI → GitHub → Vercel

제출: 프롬프트 기록 · 변경 diff · GitHub commit · 실제 Web URL